

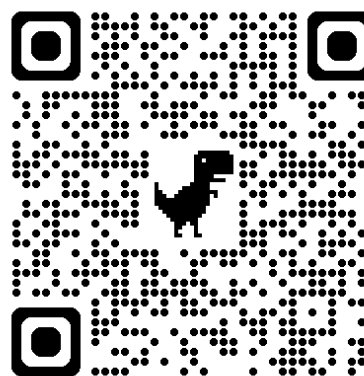


自然語言處理與應用

Natural Language Processing and Applications

遞迴神經網路與長短期
記憶網路

Instructor: 林英嘉 (Ying-Jia Lin)
2026/03/09



GitHub



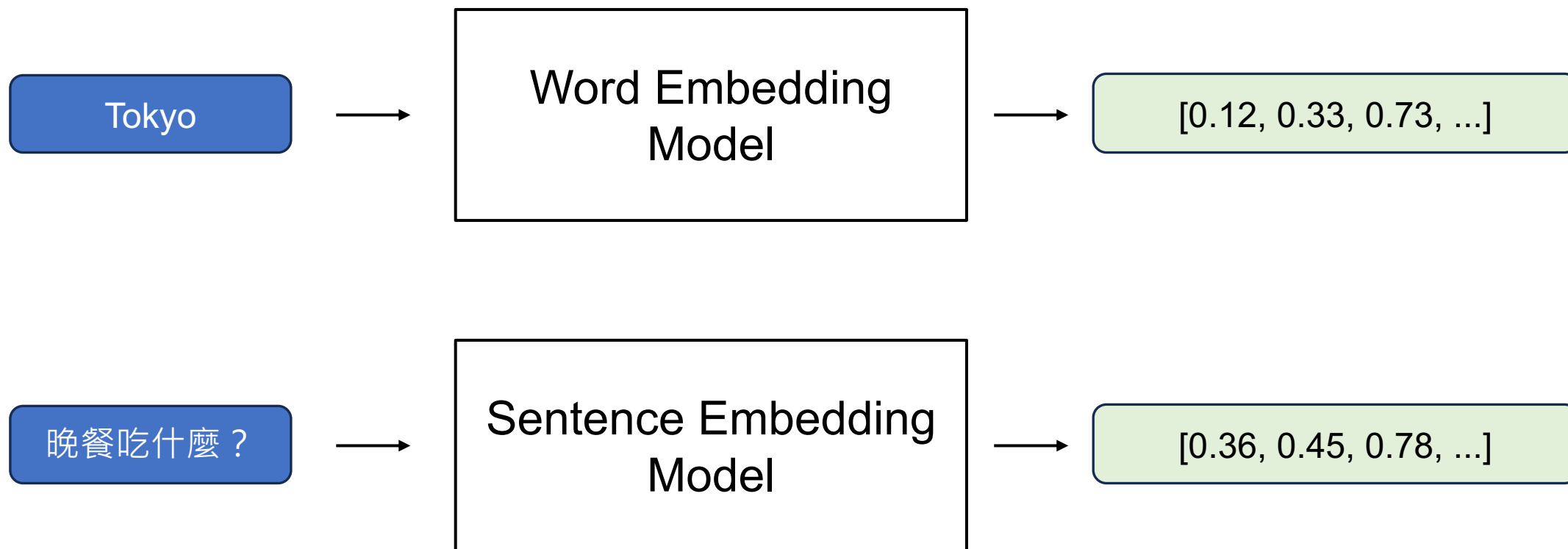
Slido
NLP0309
([Link](#))

Outline

- Recurrent Neural Networks (RNN) [100 min]
- RNN code [50 min]
- Quiz [20 min]

Recap (Embeddings)

詞向量 vs. 句向量



Google Search vs. 向量化



NLP 的 Embeddings: 語意資訊被嵌入到 (高維度) 向量空間

Bag-of-words Model

Meaning: Each **document** (**sentence**) carries a bag of words.

Bag of words (BoW)

Very good drama although it appeared to have a few blank areas leaving the viewers to fill in the action for themselves. I can imagine life being this way for someone who can neither read nor write. This film simply smacked of the real world: the wife who is suddenly the sole supporter, the live-in relatives and their quarrels, the troubled child who gets knocked up and then, typically, drops out of school, a jackass husband who takes the nest egg and buys beer with it. 2 thumbs up... very very very good movie.



('the', 8),
(',', 5),
('very', 4),
('.', 4),
('who', 4),
('and', 3),
('good', 2),
('it', 2),
('to', 2),
('a', 2),
('for', 2),
('can', 2),
('this', 2),
('of', 2),
('drama', 1),
('although', 1),
('appeared', 1),
('have', 1),
('few', 1),
('blank', 1)

.....

Figure source: <https://sfhsu29.medium.com/nlp-入門-1-text-classification-sentiment-analysis-極簡易情感分類器-bag-of-words-naive-bayes-e40d61de9a7f>

TF-IDF 為什麼這樣設計？

想像你撿到了班上三位同學的日記：

日記 A (小明的)：「我 今天 去 吃 漢堡，漢堡 超 好吃。」

日記 B (小華的)：「我 今天 去 學校 上課，好 累。」

日記 C (小美的)：「我 今天 跟 媽媽 去 買 菜。」

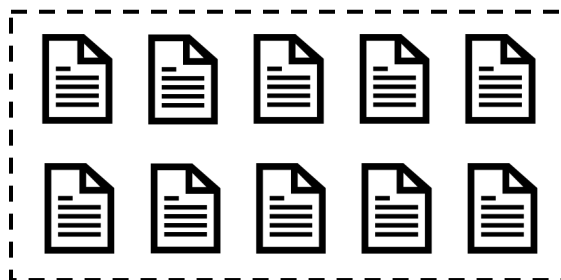
日記 A 裡的內容觀察：

單字	TF (在這篇很多?)	IDF (在別篇很少?)	結果 (TF-IDF)	意義
我	是 (1次)	否 (每句都有)	低分	雜訊/不重要
今天	是 (1次)	否 (每句都有)	低分	雜訊/不重要
漢堡	是 (2次)	是 (別句都沒提)	高分	關鍵字！

詞嵌入：把詞嵌入到向量 (Word2Vec)

[1] Mikolov et al. "Distributed representations of words and phrases and their compositionality." NeurIPS 2013.

非常大量的語料庫
(例如：Wikipedia)



Input

Steve

Jobs

[MASK]

Apple

Inc.

Word2Vec
(CBOW)

Prediction

founded

[MASK]

[MASK]

founded

[MASK]

[MASK]

Word2Vec
(Skip-gram)

Steve

Jobs

Apple

Inc.

Advanced Techniques of Word2vec

- Speed up **the training process** of Word2vec
- Hierarchical Softmax
- Negative Sampling

詞向量視覺化



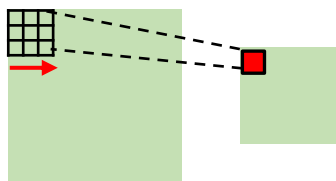
- **Word embedding model:** glove-wiki-gigaword-100
- **Dimension reduction:** t-SNE
- **Dataset:** Mikolov et al., 2013

Try it:
<https://www.cs.cmu.edu/~dst/WordEmbeddingDemo/>

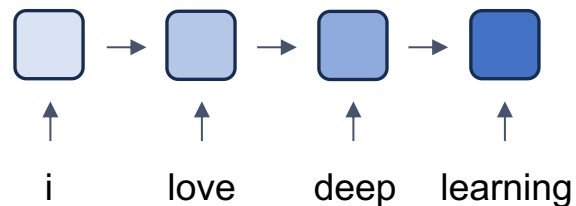
RNNs (Recurrent Neural Networks)

深度學習常見模型架構

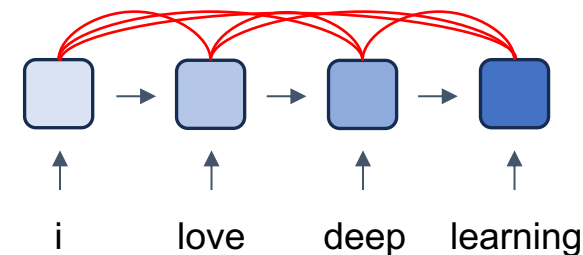
CNN (著重局部資訊)



RNN (著重記憶力)



Transformer (著重全局資訊與記憶力)



NLP 三大秘辛

1 每個字，都是一段向量

2 每個 NLP 模型都有一個字典 

3 自然語言處理模型需要記憶力 

nn.Embeddings

```
1 word_to_idx = {"好吃": 0, "棒": 1, "给力": 2}
2 embeds = torch.nn.Embedding(3, 5) # 2 words in vocab, 5 dimensional embeddings
```

```
1 lookup_tensor = torch.tensor(
2     [
3         word_to_idx["好吃"],
4         word_to_idx["棒"],
5         word_to_idx["给力"],
6     ],
7     dtype=torch.long,
8 )
9 word_embed = embeds(lookup_tensor)
10 print(word_embed)
```

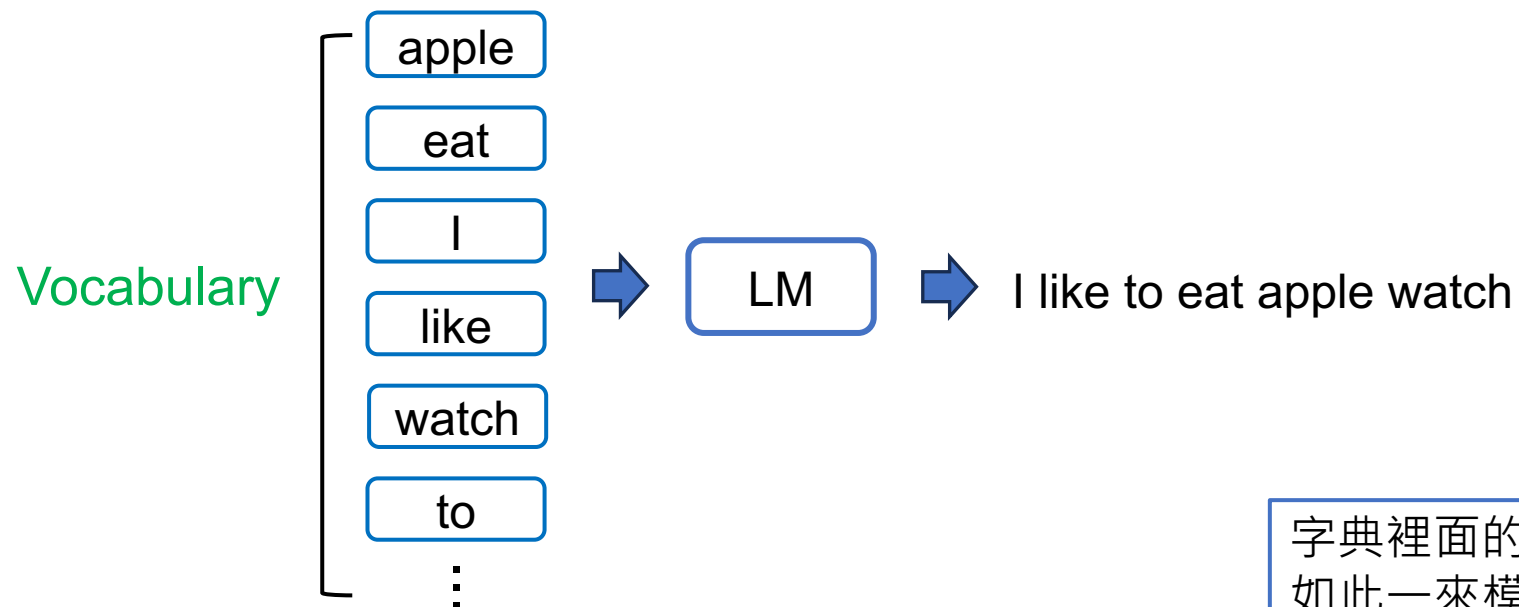
[7]

```
... tensor([[ 2.2014,  1.9880,  1.5726, -0.1552,  0.0475],
          [-0.0540, -1.5926, -0.5808, -0.6631, -0.3321],
          [-0.3562,  1.0019,  0.8178, -0.9174, -0.0707]],
        grad_fn=<EmbeddingBackward0>)
```

NLP 三大秘辛

- 1 每個字，都是一段向量
- 2 每個 NLP 模型都有一個字典 
- 3 自然語言處理模型需要記憶力 

每個 NLP 模型都有一個字典



字典裡面的內容是語言模型能理解的最小單元，
如此一來模型才能夠進行目標任務 (e.g., 生成)



... (你平常操作 ChatGPT 的結果)

字典實作

字典的變數
(key: 字, value: index)

通常我們會預先準備 <pad> 跟 <unk>
前者代表 padding，後者代表 unknown words

```
1 vocab = {'<pad>':0, '<unk>':1}
2
3 # 使字詞不重複
4 words = sorted(set(words))
5 for idx, word in enumerate(words):
6     # 一開始已經放兩個進去 dictionary 了
7     idx = idx + 2
8     # 將 word to id 放到 dictionary
9     vocab[word] = idx
```

Python

字典在NLP中的實作方式就真的採用Python dictionary

[注意事項] 字典實作

字典是經由訓練資料集的字所建立的

```
word_to_idx = {"<pad>": 0, "<unk>": 1, "好吃": 2, "棒": 3, "给力": 4}
embeds = torch.nn.Embedding(5, 5) # 5 words in vocab, 5 dimensional embeddings
```

```
def get_word_id(word, vocab, unk_idx: int = 1):
    return vocab.get(word, unk_idx)
```

如果某字不存在於字典中，則通常給予 <unk> 的 id

```
lookup_tensor = torch.tensor([
    get_word_id("好吃", word_to_idx),
    get_word_id("不棒", word_to_idx),
    get_word_id("<unk>", word_to_idx),
])
word_embed = embeds(lookup_tensor)
print(word_embed)

tensor([[ -0.5574,  0.1768,  0.9588,  1.2837, -1.7256],
        [-3.4725, -1.0435, -0.0538, -0.0625, -0.4173],
        [-3.4725, -1.0435, -0.0538, -0.0625, -0.4173]]
grad_fn=<EmbeddingBackward0>)
```

NLP 三大秘辛

- 1 每個字，都是一段向量
- 2 每個 NLP 模型都有一個字典 
- 3 自然語言處理模型需要記憶力 

語言的模式

★ 上下文非常重要

句子1：

小明昨天遇到小美，他對她說：「下次一起去看電影吧！」
，請問小明下次想去跟誰看電影？

句子2：

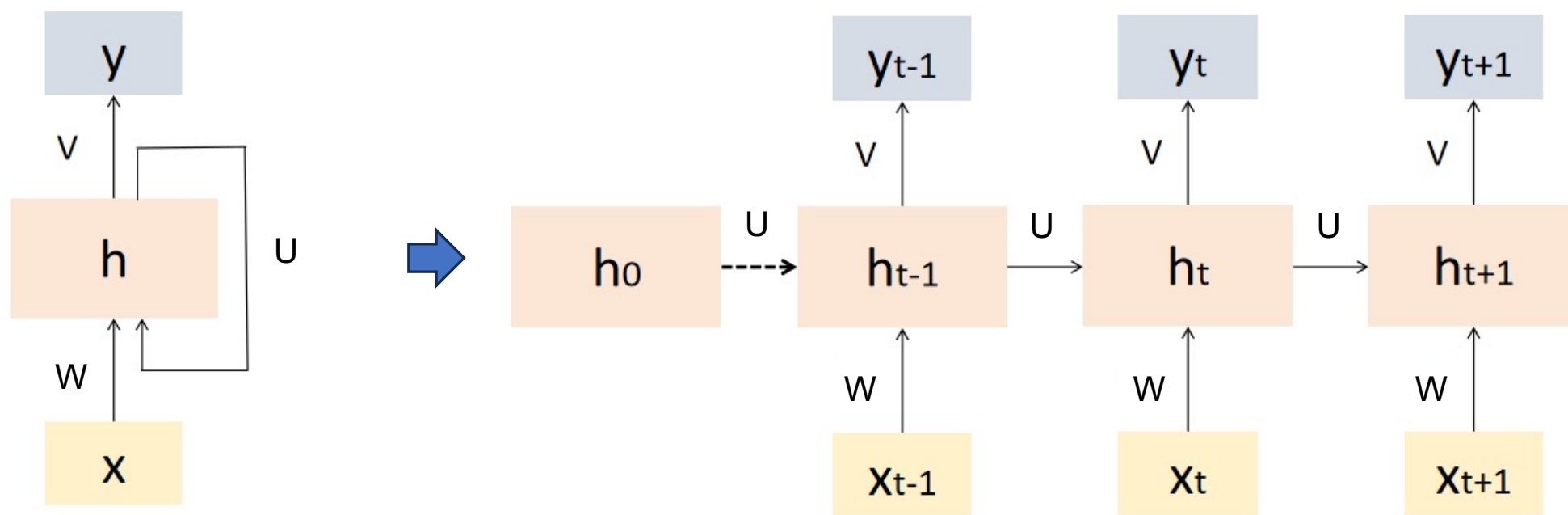
「庭院深深深幾許」，請問有幾個深？

序列模型

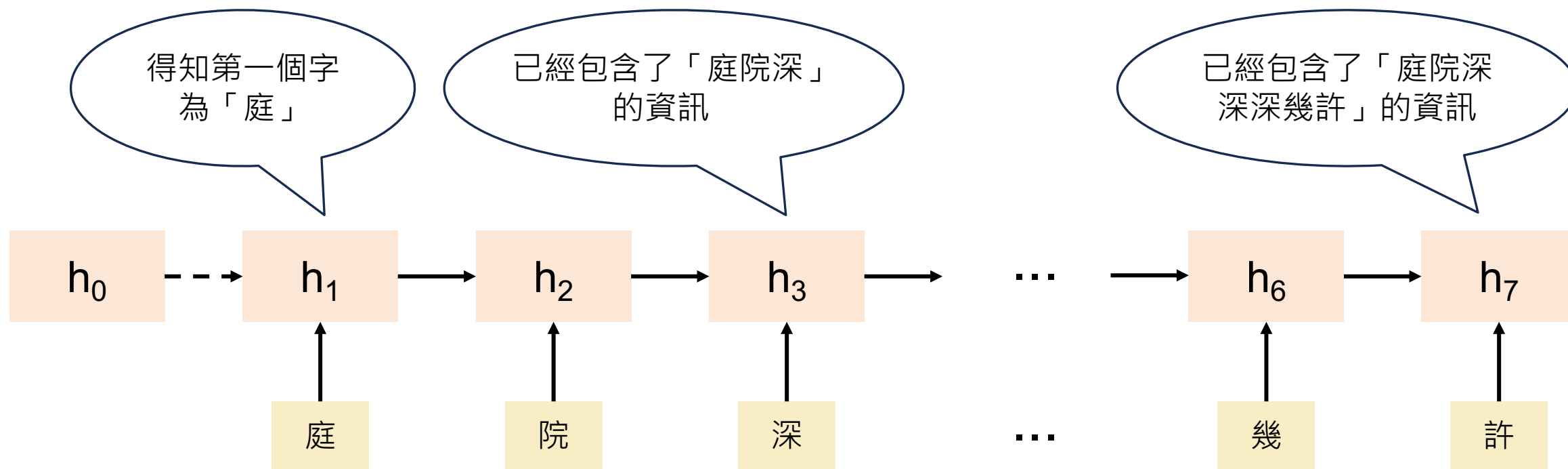
RNN

Recurrent Neural Networks (RNN)

- Recurrent Neural Networks (RNNs) are a type of artificial neural network designed to handle sequential data by **capturing temporal dependencies**.
 - RNN 的“遞迴”是指它在每一個時間步中重複使用相同的參數（同一組權重），並把先前的隱藏狀態傳到下一個時間步



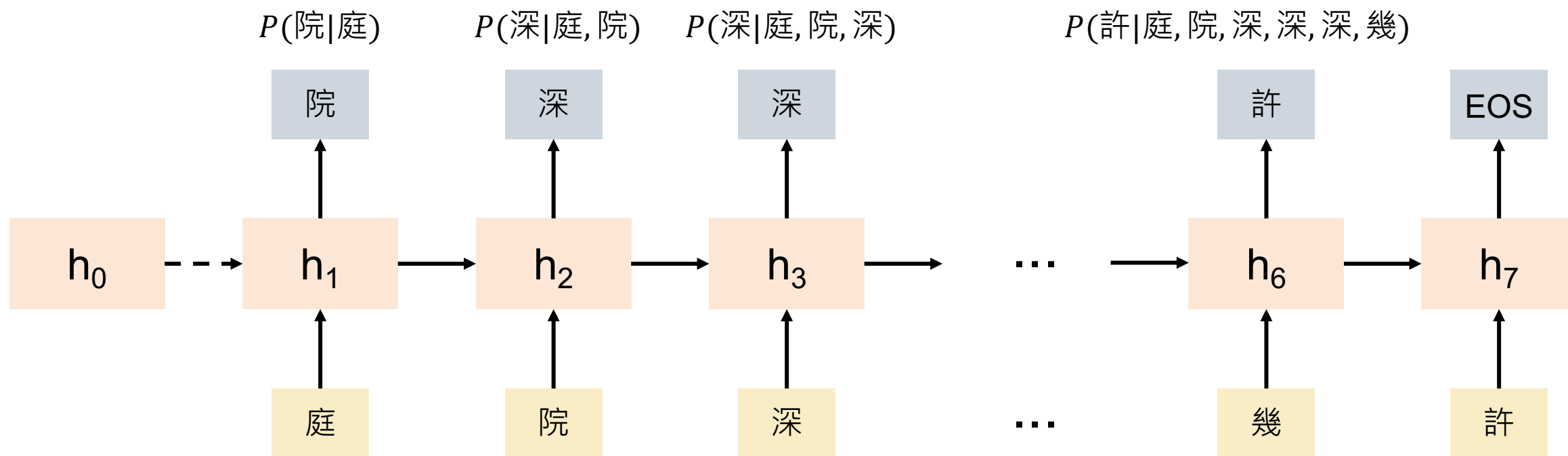
RNN 的記憶力意義



RNN 以文字接龍進行訓練

EOS: End of Sentence

$P(\text{EOS}|\text{庭, 院, 深, 深, 深, 幾, 許})$



目標函數： $\log(P(\text{院}|\text{庭}) \times P(\text{深}|\text{庭, 院}) \times P(\text{深}|\text{庭, 院, 深}) \times \dots \times P(\text{EOS}|\text{庭, 院, 深, 深, 深, 幾, 許}))$

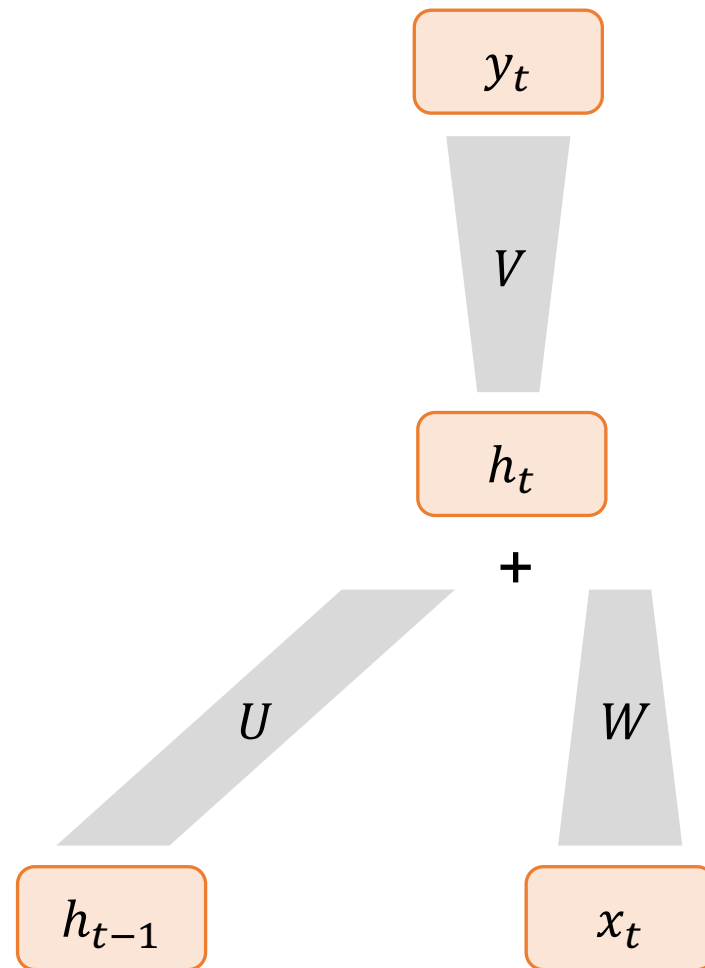
RNN (本頁省略 NN 的 bias)

➤ The equation on step t is:

$$h_t = \tanh(Wx_t + Uh_{t-1})$$

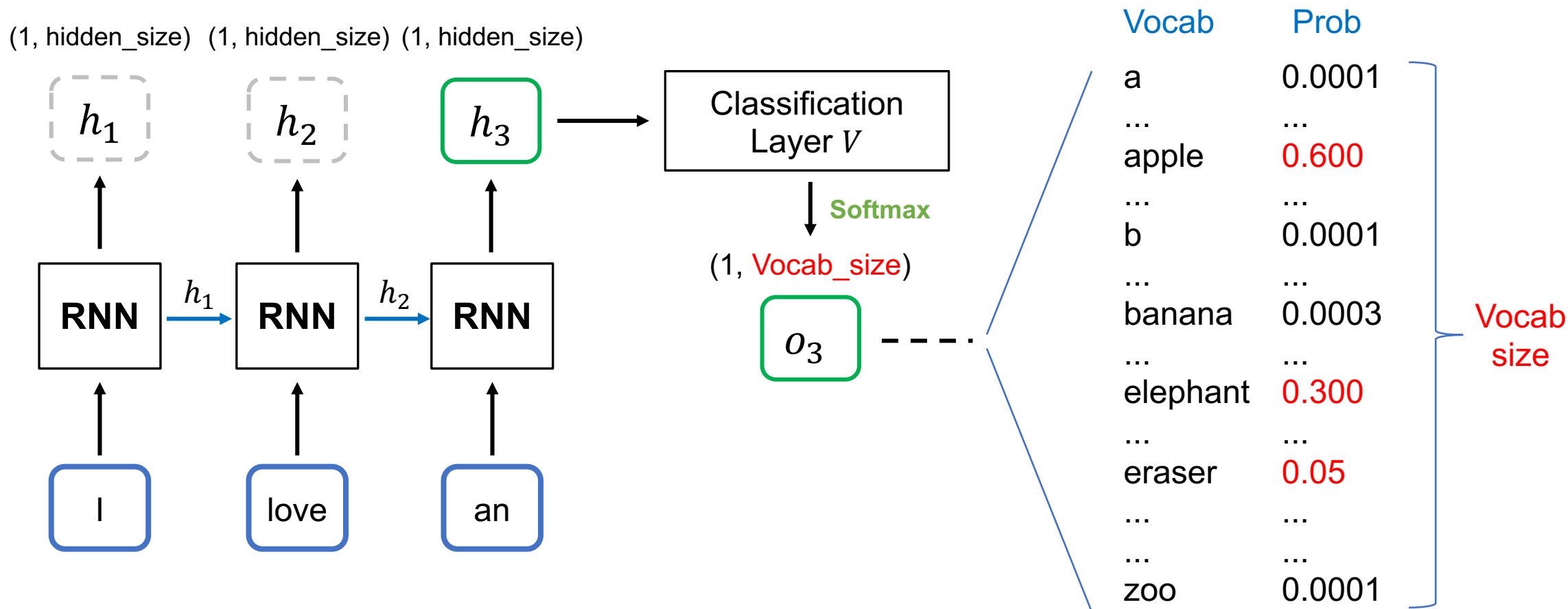
$$y_t = Vh_t$$

, where $\tanh$ is an activation function.



Text Generation with RNN

*本頁的RNN不包含輸出層 V



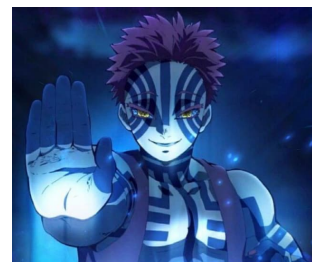
Why is softmax called softmax?

soft: 機率 (帶有資訊量)

Max:



1

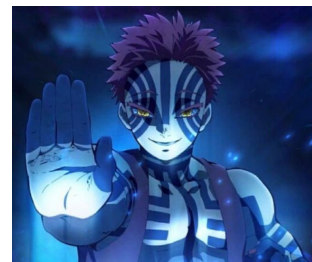


0

Softmax:



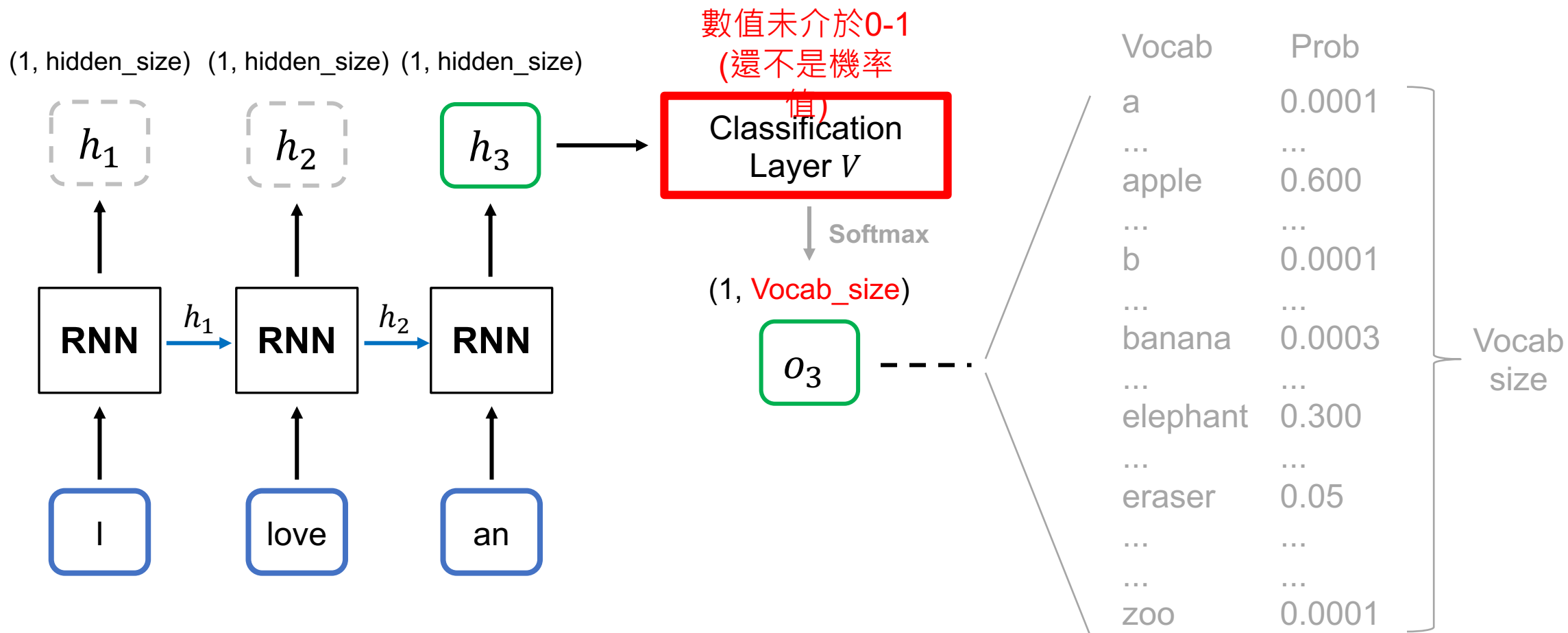
0.6



0.4

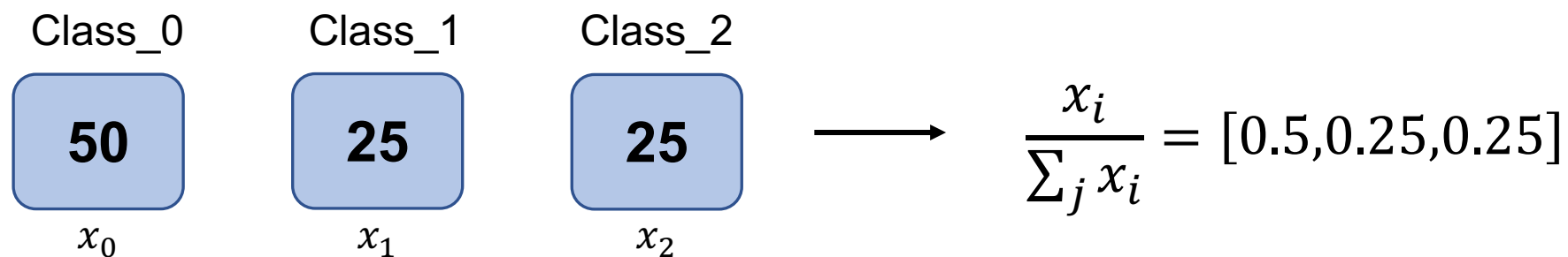
Text Generation with RNN (V 輸出時)

*本頁的RNN不包含輸出層 V



如何把模型輸出變成機率值 (0~1) ?

機率總和為1，每個機率介於 0 ~ 1 之間

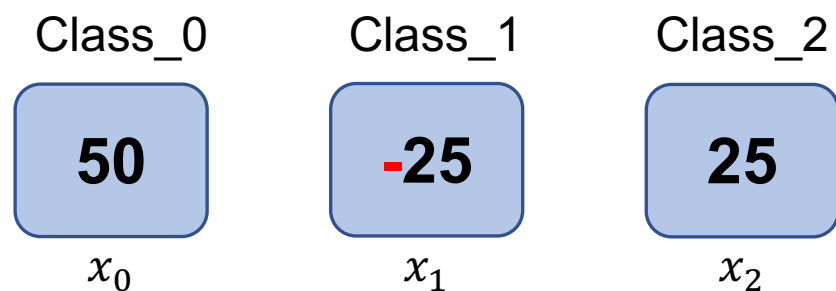


- **Softmax function**，採用 exponential: $\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$
 1. 恆正 (符號不變性)
 2. 大的數值更大，小的數值更小，有助於梯度下降

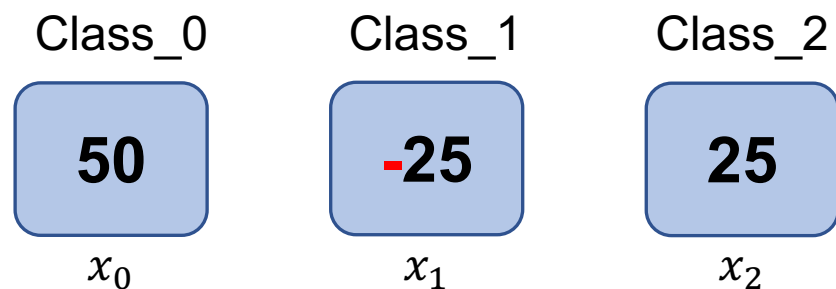
注意！Softmax 有考慮到類別總和

為什麼恆正重要？

機率總和為1，每個機率介於 0 ~ 1 之間



$$\frac{x}{\sum_j x_j} = [1.0, -0.5, 0.5]$$



$$\begin{aligned}\text{Softmax}(x_i) &= \frac{e_i^x}{\sum_j e^{x_j}} \\ &= [1.0, 2.6786e - 33, 1.3888e - 11]\end{aligned}$$

Softmax 帶來大者恆大的效應

```
import torch
from torch.nn.functional import softmax

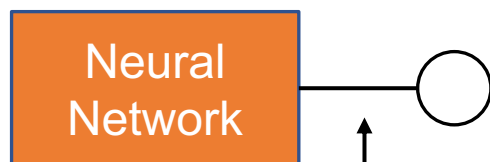
logits = torch.tensor([50, 25, 25], dtype=torch.float32)
print(softmax(logits, dim=0))
```

```
tensor([1.0000e+00, 1.3888e-11, 1.3888e-11])
```

```
# logits = torch.tensor([5, 2, 2], dtype=torch.float32)
tensor([0.9094, 0.0453, 0.0453])
```

如何對一個模型輸出值取機率值 (0~1) ?

- Sigmoid function:



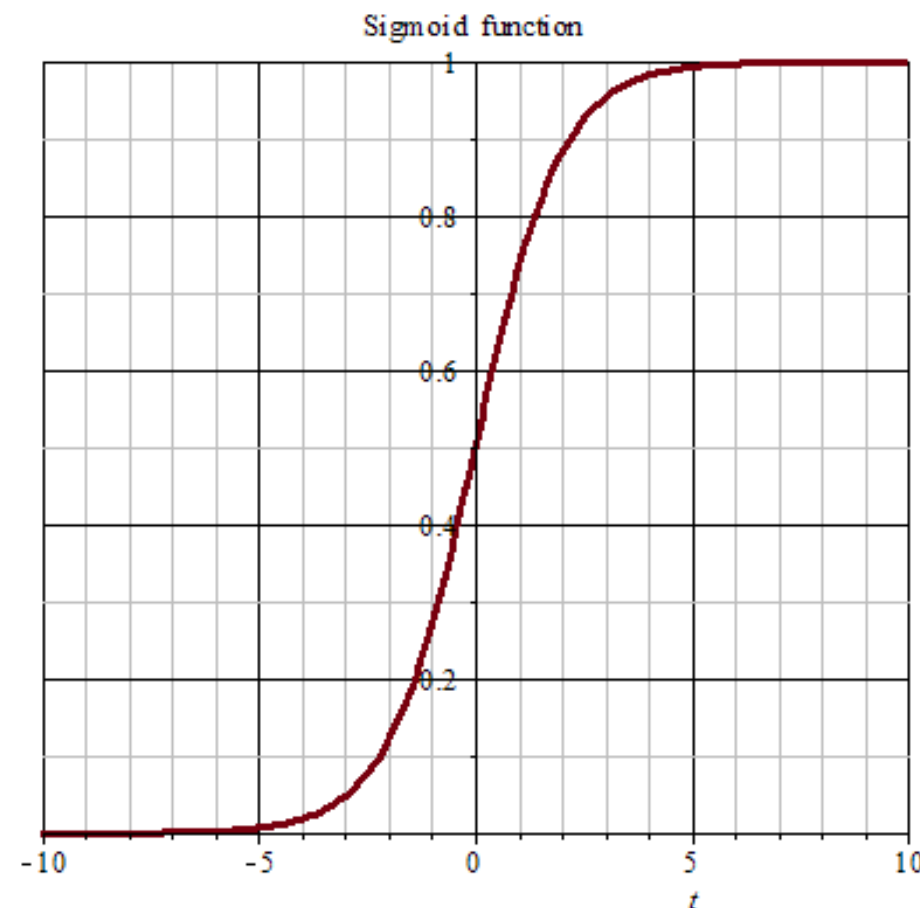
$$\text{Sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

假設閾值為0.5

> 0.5 是貓，
否則不是貓

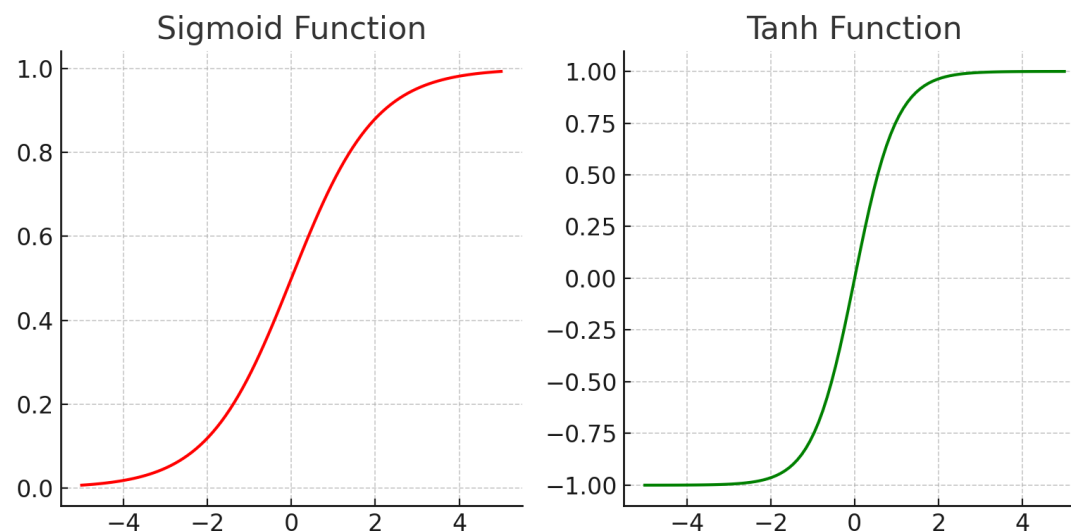
x代表logit
x很大代表 e^{-x} 很小
→ Sigmoid(x)很大

注意！sigmoid 只需要1個數值就能計算



RNN 使用 tanh (hyperbolic tangent)

$$\tanh(x) = \frac{\sinh(x)}{\cosh(x)} = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



RNN 經常使用 tanh 的原因：

RNN 作為序列模型，需要在多個時間步維持梯度，tanh 的數值在 -1 到 1 之間，有助於梯度不會太大或太小 (Sigmoid 的數值是 0 到 1 之間)

Activation functions (啟動函數)

	值的範圍	使用情境
softmax	[0, 1] (sum up to 1)	需要讓多個值產生相加為一的機率值的時候，常用於建立分類任務模型時計算 cross-entropy loss
sigmoid	[0, 1]	需要讓單一值產生機率值的時候，常用於建立二元分類任務模型時計算 binary cross-entropy loss
tanh	[-1, 1]	常用於LSTM等RNN模型中，主要為引入非線性轉換。tanh的值是對稱的，以零為中心，在有些訓練條件下和sigmoid相比較不會梯度消失
ReLU	[0, 正值]	常用於CNN或MLP等模型中，主要為引入非線性轉換。計算簡單，在許多任務收斂快

為什麼我們通常不用 MLP 來做 NLP ?

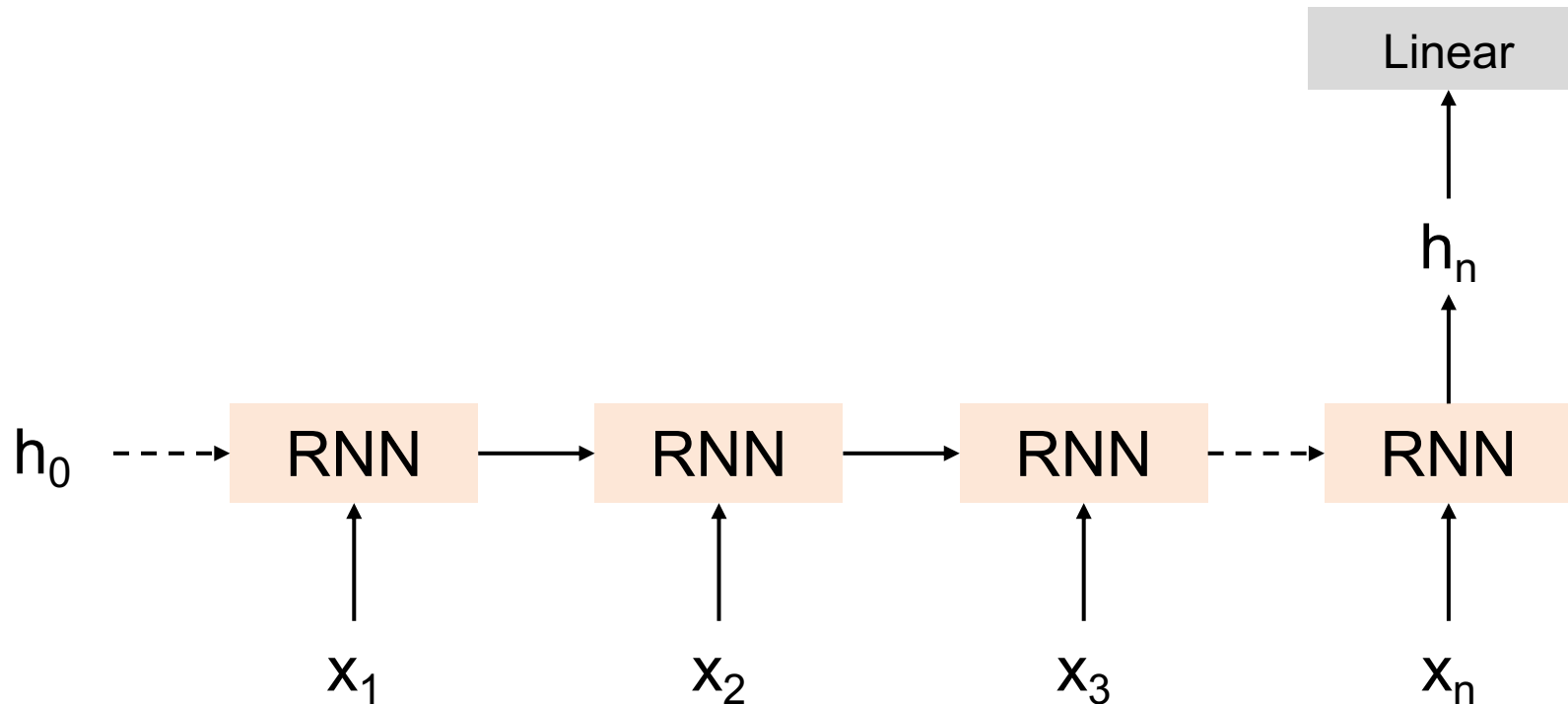
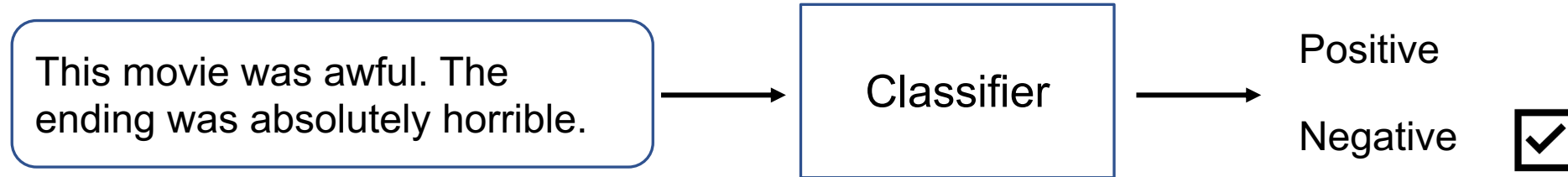
1. 無法處理序列順序

- MLP 不會記住序列的前後順序 (上下文關係)，但語言的順序非常重要。

2. 輸入長度必須固定

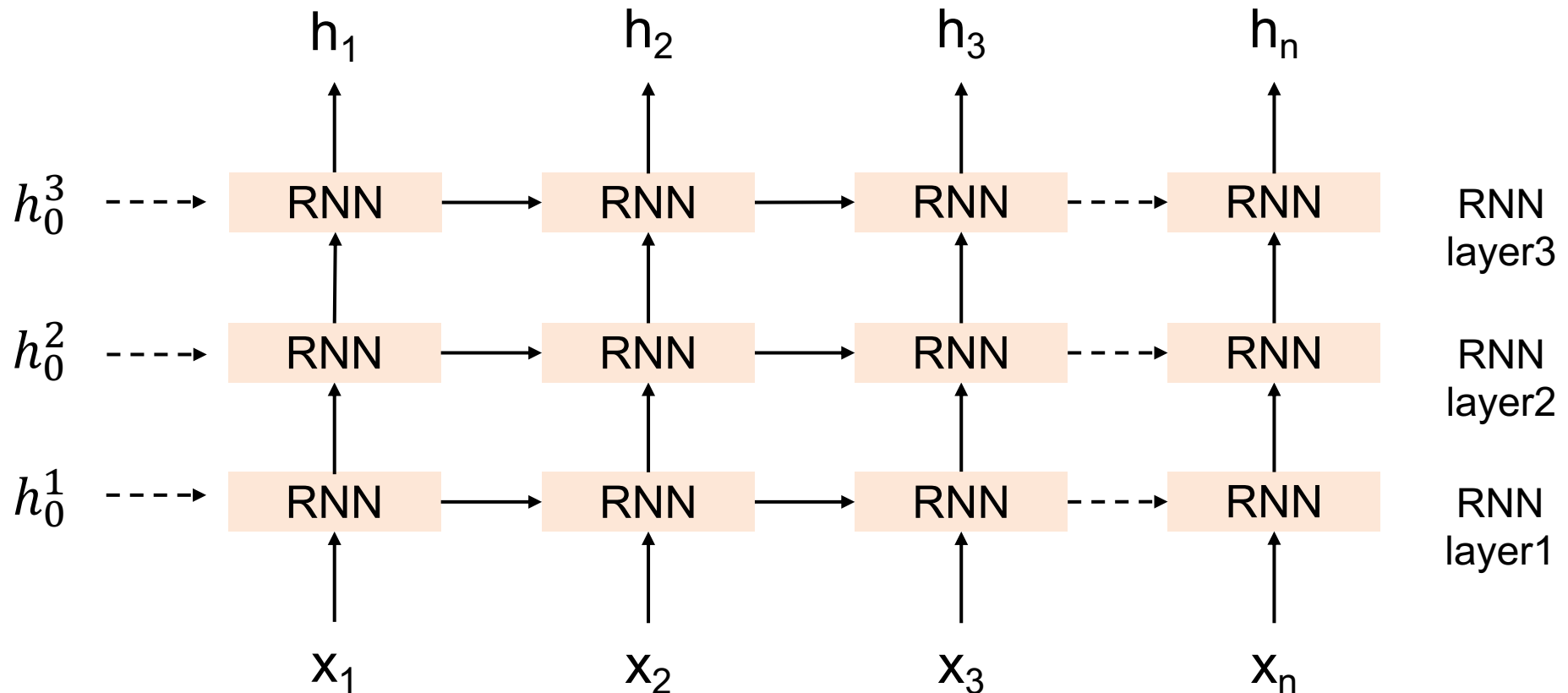
- MLP 需要固定長度的輸入，但句子的長度通常不同。

RNNs for Sequence Classification



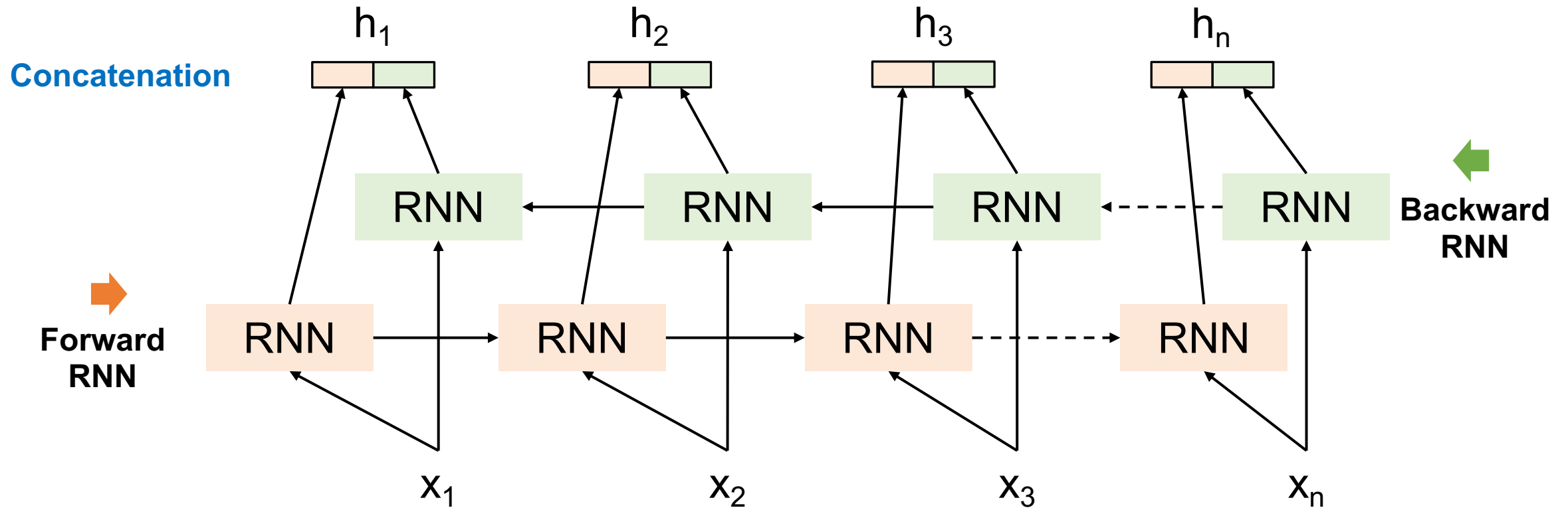
Stacked RNNs

- Stacked RNNs consist of multiple networks where the output of one layer serves as the input to a subsequent layer



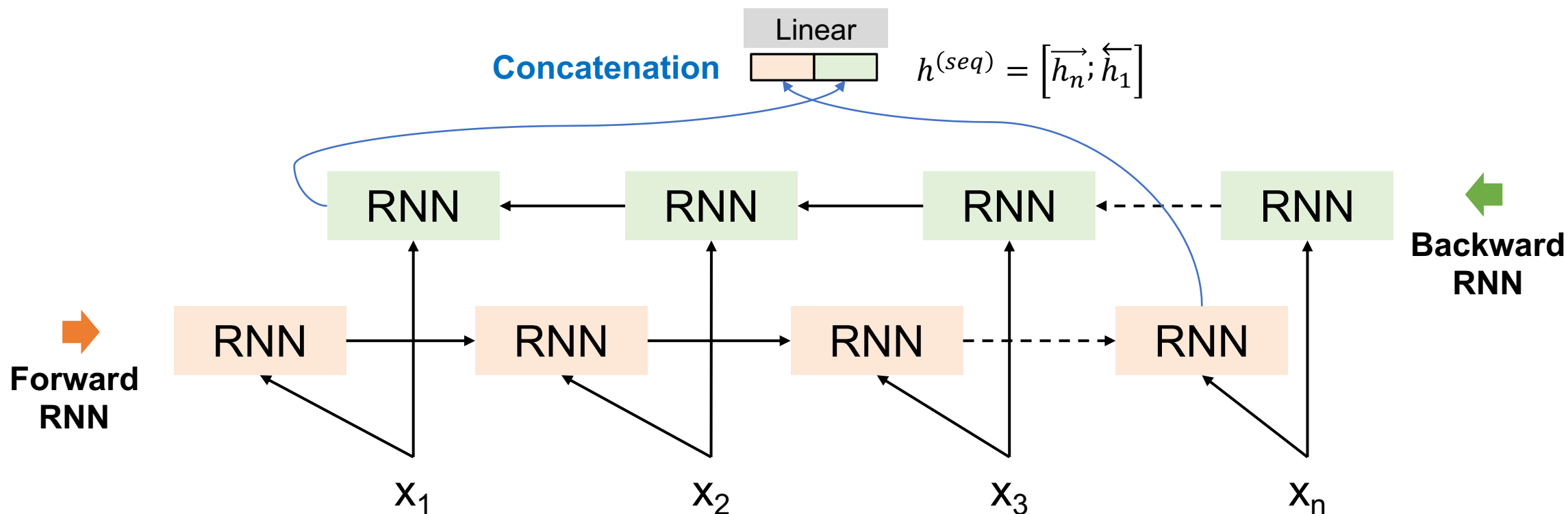
Bidirectional RNNs

$$h_t = [\vec{h}_t; \overleftarrow{h}_t]$$



Bidirectional RNNs for Sequence Classification

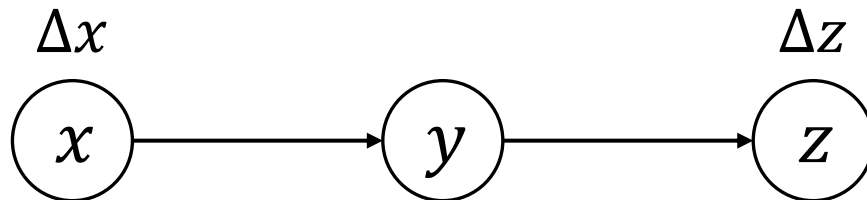
- The final hidden units from the forward and backward passes are combined to represent the entire sequence.
- This combined representation serves as input to the subsequent classifier.



Shortage of standard RNNs

- Gradient vanishing (梯度消失) problem
- Long-term dependencies cannot be handled.
 - RNNs can only remember local relationships.
 - This is result from **back-propagation through-time (BPTT)** during training.

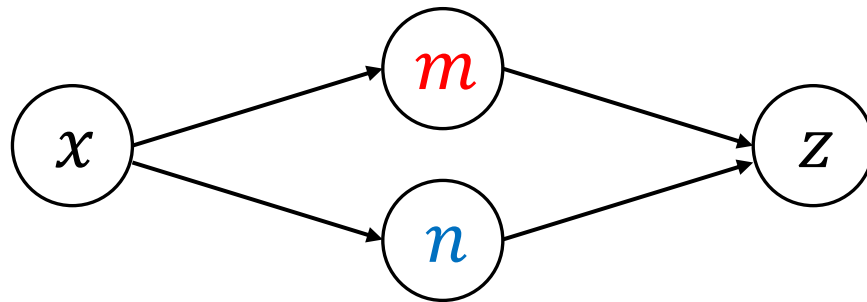
(Calculus) Chain Rule - 1



$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}$$

- $\frac{dz}{dx}$ 代表 x 對 z 的影響，但 x 的變化會影響到 y ，接著 y 影響到 z

(Calculus) Chain Rule - 2



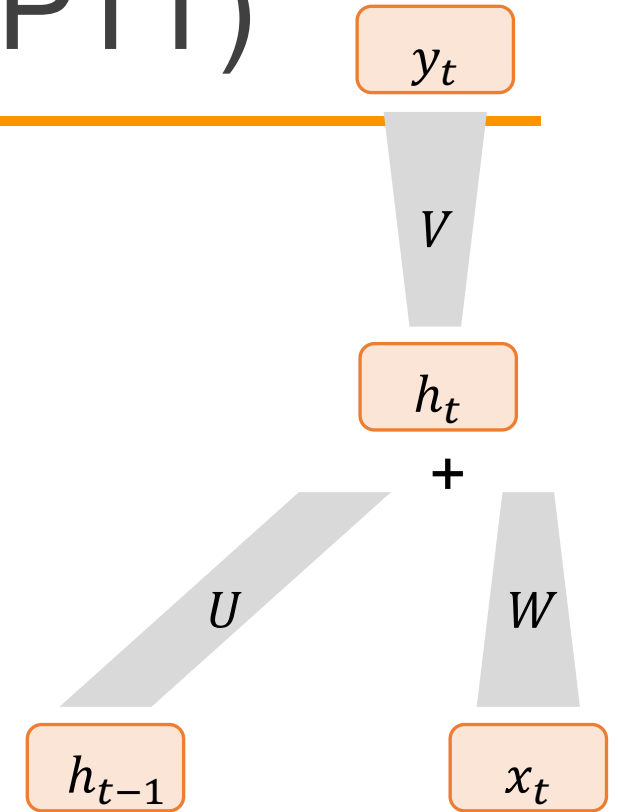
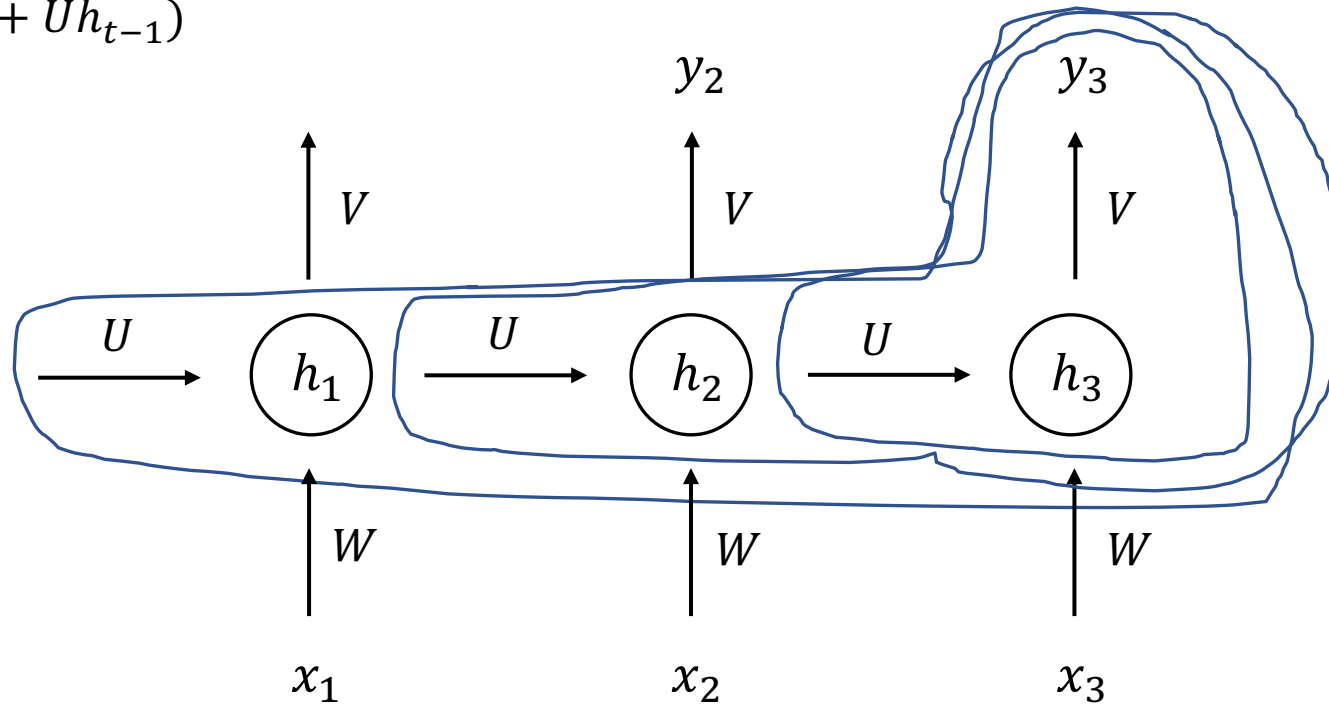
$$\frac{dz}{dx} = \frac{dz}{dm} \frac{dm}{dx} + \frac{dz}{dn} \frac{dn}{dx}$$

- $\frac{dz}{dx}$ 代表 x 對 z 的影響
 - 但 z 的變化由 m 和 n 共同影響

Back-propagation through-time (BPTT)

$$h_t = \tanh(Wx_t + Uh_{t-1})$$

$$y_t = Vh_t$$



$$\frac{\partial \mathcal{L}}{\partial U} = \frac{\partial \mathcal{L}}{\partial y_3} \cdot \frac{\partial y_3}{\partial h_3} \cdot \frac{\partial h_3}{\partial U} + \frac{\partial \mathcal{L}}{\partial y_3} \cdot \frac{\partial y_3}{\partial h_3} \cdot \frac{\partial h_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial U} + \frac{\partial \mathcal{L}}{\partial y_3} \cdot \frac{\partial y_3}{\partial h_3} \cdot \frac{\partial h_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial U}$$

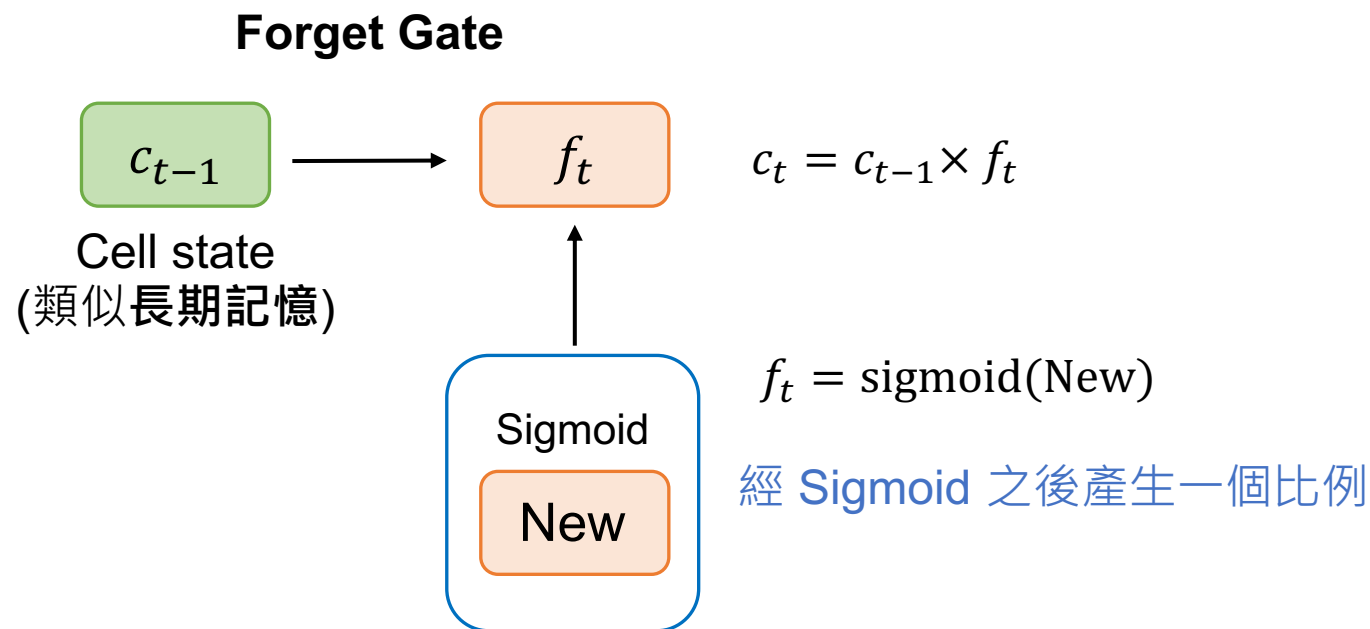
(small)

Long Short-Term Memory

LSTM

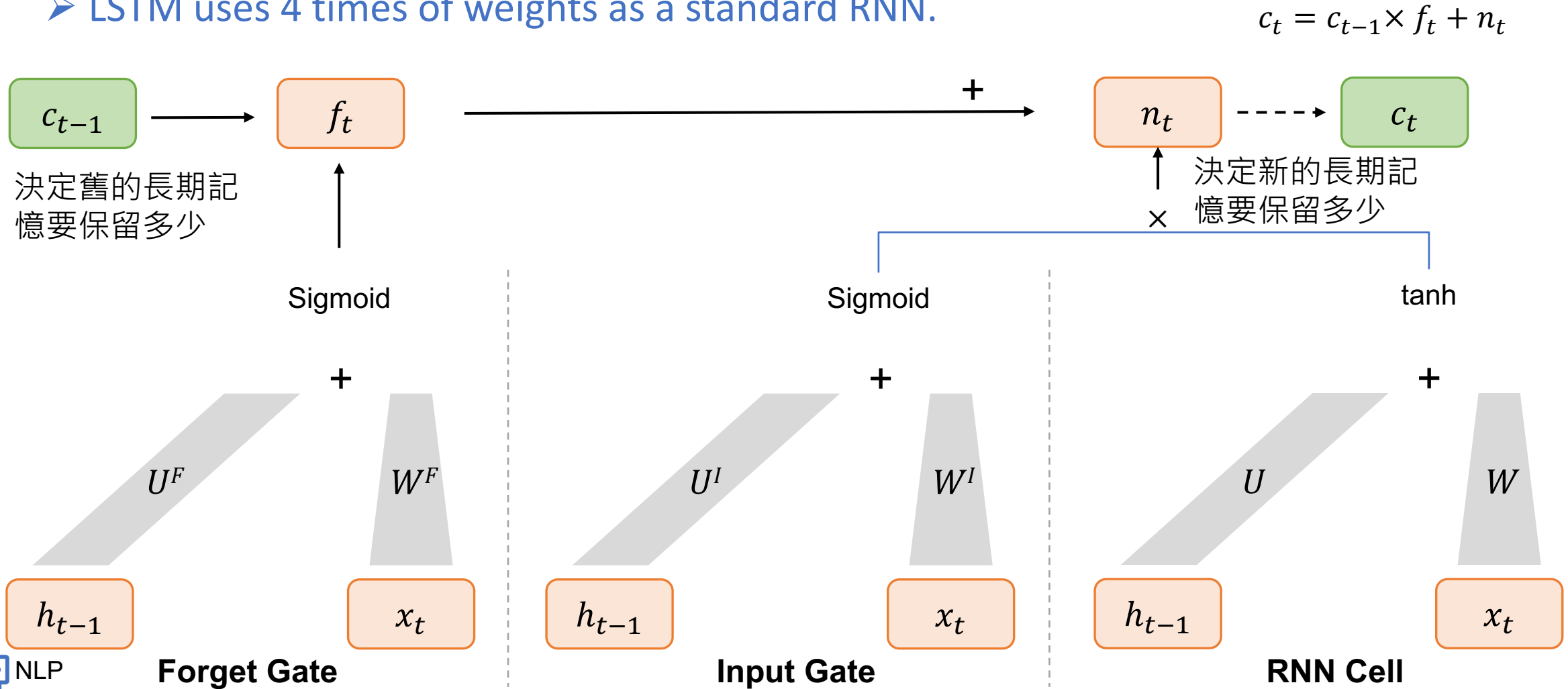
Gate (以 Forget Gate 為例)

LSTM 新增了 Cell State



LSTM (Long Short-Term Memory)

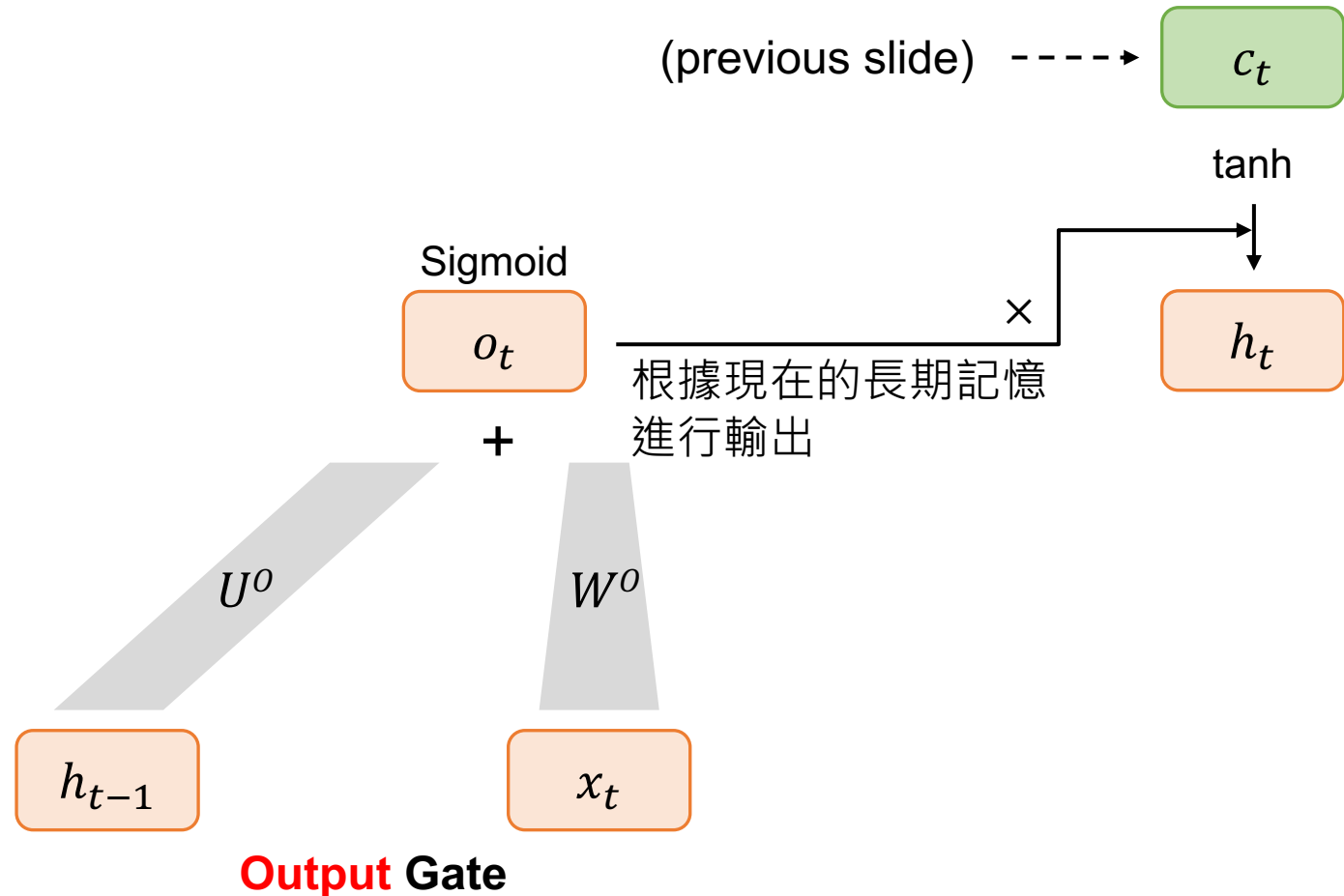
➤ LSTM uses 4 times of weights as a standard RNN.



LSTM (Long Short-Term Memory)

- LSTM uses 4 times of weights as a standard RNN.

$$c_t = c_{t-1} \times f_t + n_t$$



延伸學習

<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

RNN vs. LSTM

	Memory Mechanism	Handles Long Dependencies?	Gradient Vanishing?
RNN	Hidden state h_t	Worse	Yes
LSTM	Cell state c_t + Hidden state h_t	Better	Much less than a standard RNN

Thank you!

Instructor: 林英嘉

 yjlin@cgu.edu.tw

TA: 林宣辰

 b1128015@cgu.edu.tw